Speech Runtime

Purpose

Speech Runtime is responsible for verbal communication between the digital character and the outside world.

Its responsibility is not limited to speech recognition or speech synthesis.

Its purpose is to manage the complete conversational pipeline, transforming incoming communication into meaningful interaction and transforming the character's responses back into spoken language.

Speech Runtime is the communication system of the character.

It allows Shio to hear, understand, think, speak and participate in conversations while remaining completely independent from rendering, external services and visual presentation.

Responsibilities

Speech Runtime owns every stage of spoken communication.

Its responsibilities include:

- speech recognition (ASR);
- voice activity detection (VAD);
- intent routing;
- Rule Gate evaluation;
- LLM interaction;
- streaming response generation;
- sentence chunking;
- speech synthesis (TTS);
- speech queue management;
- interruption handling;
- conversation flow control.

Speech Runtime transforms audio into understanding and understanding into speech.

Its responsibility ends when spoken output has been produced and the Character State has been updated.

Non-Responsibilities

Speech Runtime deliberately does **not** own:

- avatar animations;
- facial expressions;
- gestures;
- body language;
- memory persistence;
- tool execution;
- platform integrations;
- moderation logic;
- rendering;
- overlays.

These responsibilities belong to other Runtime systems.

Maintaining these boundaries is essential for preserving architectural independence.

Internal Pipeline

Although implementation details may evolve, Speech Runtime conceptually follows the same communication pipeline.

```
Audio Input

↓

Voice Activity Detection

↓

Automatic Speech Recognition

↓

Intent Router

↓

Rule Gate

↓

LLM

↓
```

```
Streaming Response

↓

Sentence Chunker

↓

TTS Queue

↓

Audio Output
```

Each stage performs one responsibility.

No stage should become responsible for unrelated concerns.

Streaming First

Speech Runtime is designed around streaming rather than batch processing.

The Runtime should begin producing output as soon as meaningful information becomes available.

Examples include:

- partial ASR recognition;
- streaming LLM generation;
- sentence-level TTS;
- incremental playback.

Streaming minimizes perceived latency and creates a more natural conversational experience.

Responsiveness is preferred over waiting for complete responses.

Intent Router

Not every piece of recognized speech should reach the LLM.

The Intent Router classifies incoming input into high-level categories.

Examples include:

- normal conversation;
- direct command;

- system command;
- ignored input;
- future passive conversation.

Intent classification remains lightweight.

Its purpose is routing rather than reasoning.

Rule Gate

Rule Gate is the first decision layer of Speech Runtime.

Its purpose is to determine whether recognized input deserves further processing.

Simple deterministic rules should always be preferred over unnecessary LLM calls.

Typical Rule Gate decisions include:

- ignoring background speech;
- filtering duplicate recognition;
- detecting wake phrases;
- rejecting extremely short fragments;
- identifying explicit commands.

Only uncertain situations should require additional reasoning.

This minimizes latency and computational cost while preserving intelligent behavior.

Optional LLM Classification

Some conversational situations cannot be reliably resolved using deterministic rules alone.

In these rare cases, Rule Gate may delegate classification to a lightweight language model.

This model does **not** generate replies.

It only classifies intent.

Typical questions include:

- Is Shio being addressed?
- Should this message be ignored?
- Is this part of the ongoing conversation?
- Does this require a response?

Its purpose is reducing uncertainty rather than replacing deterministic routing.

This classification layer should be invoked only when Rule Gate cannot confidently make a decision.

Sentence Chunker

The LLM should not wait until the entire response has been generated.

Speech Runtime divides streamed text into complete sentences.

Each sentence becomes independently available for speech synthesis.

This architecture allows speech generation to overlap with ongoing language generation.

The result is significantly lower perceived response latency.

TTS Queue

Speech Runtime owns the speech queue.

The queue guarantees:

- sequential playback;
- interruption handling;
- cancellation;
- future prioritization;
- synchronization with Character State.

Speech synthesis should never directly control playback.

Playback remains the responsibility of the Runtime.

Character State Integration

Speech Runtime contributes to multiple Character State dimensions.

Examples include:

- Runtime Mode;
- Speech State;
- Activity;
- Attention.

Speech Runtime never directly controls Avatar Runtime.

Instead, Avatar Runtime observes Character State and reacts independently.

Future Evolution

Speech Runtime is expected to evolve without changing its architectural responsibility.

Potential future improvements include:

- multilingual recognition;
- multiple simultaneous microphones;
- streaming translation;
- emotion-aware speech generation;
- passive listening mode;
- contextual interruption recovery.

Regardless of future capabilities, Speech Runtime will remain responsible for one thing only:

communication between the character and the outside world.